# Day 8 – Async Webhooks with Celery

**Goals:** Move webhook delivery off the main thread using Celery + Redis, add automatic retries, and log every delivery attempt.

---

### Step 1 — Run Redis

```
docker run -d -p 6379:6379 redis:7-alpine
```

---

### Step 2 — Configure Celery

`gamekey_platform/celery.py`:

```
import os
from celery import Celery

os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'gamekey_platform.setti

app = Celery('gamekey_platform')
app.config_from_object('django.conf:settings', namespace='CELERY')
app.autodiscover_tasks()
```

Update `gamekey_platform/__init__.py` so Django loads Celery on startup:

```
from .celery import app as celery_app

__all__ = ('celery_app',)
```

Add to `settings.py`:

```
CELERY_BROKER_URL = 'redis://localhost:6379/0'
CELERY_RESULT_BACKEND = 'redis://localhost:6379/0'
CELERY_TASK_ALWAYS_EAGER = False  # set True in tests to run tasks sync
```

---

### Step 3 — Webhook Delivery Log Model

Add to `games/models.py`:

```
class WebhookDeliveryLog(models.Model):
    game_key = models.CharField(max_length=50)
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    payload = models.JSONField()
    response_status = models.IntegerField(null=True, blank=True)
    error_message = models.TextField(blank=True)
    attempt = models.IntegerField(default=0)
    success = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
```

```
    class Meta:
        ordering = ['-created_at']

    def __str__(self):
        status = 'OK' if self.success else 'FAIL'
        return f"[{status}] {self.game_key} attempt #{self.attempt}"
```

```
python manage.py makemigrations
python manage.py migrate
```

Register in `admin.py`:

```
from .models import WebhookDeliveryLog
admin.site.register(WebhookDeliveryLog)
```

---

## Step 4 — Celery Task with Retries

`games/tasks.py`:

```
import hmac
import hashlib
import json
import requests
from celery import shared_task
from .models import Publisher, WebhookDeliveryLog


@shared_task(bind=True, max_retries=3, default_retry_delay=60)
def send_expiry_webhook_async(self, publisher_id, game_key_str, game_ti
    publisher = None
    payload = {}

    try:
        publisher = Publisher.objects.get(id=publisher_id)

        payload = {
            "event": "game_key.expired",
            "game_key": game_key_str,
            "game_title": game_title,
            "expired_at": expired_at_iso,
            "attempt": attempt,
        }

        secret = publisher.webhook_secret.encode()
        body = json.dumps(payload, sort_keys=True).encode()
        signature = hmac.new(secret, body, hashlib.sha256).hexdigest()

        headers = {
            "Content-Type": "application/json",
            "X-Signature": f"sha256={signature}",
        }

        response = requests.post(
```

```
            publisher.webhook_url,
            json=payload,
            headers=headers,
            timeout=5,
        )

        WebhookDeliveryLog.objects.create(
            game_key=game_key_str,
            publisher=publisher,
            payload=payload,
            response_status=response.status_code,
            attempt=attempt,
            success=response.status_code < 400,
        )

        if response.status_code >= 400:
            raise Exception(f"HTTP {response.status_code} from publishe

    except Exception as exc:
        if publisher:
            WebhookDeliveryLog.objects.create(
                game_key=game_key_str,
                publisher=publisher,
                payload=payload,
                error_message=str(exc),
                attempt=attempt,
                success=False,
            )

        # Exponential backoff: 60s, 120s, 240s
        raise self.retry(exc=exc, countdown=60 * (2 ** attempt))
```

---

## Step 5 — Update Management Command

games/management/commands/check_expired_keys.py:

```
from django.core.management.base import BaseCommand
from django.utils import timezone
from games.models import GameKey
from games.tasks import send_expiry_webhook_async


class Command(BaseCommand):
    help = 'Mark expired keys and dispatch async webhook notifications.

    def handle(self, *args, **options):
        expired_keys = GameKey.objects.select_related('game__publisher'
            expires_at__lte=timezone.now(),
            status='active'
        )

        # Capture before update() clears the queryset
```

```
        keys_to_notify = list(expired_keys)
        count = expired_keys.update(status='expired')

        for key in keys_to_notify:
            send_expiry_webhook_async.delay(
                publisher_id=key.game.publisher.id,
                game_key_str=key.key_string,
                game_title=key.game.title,
                expired_at_iso=key.expires_at.isoformat(),
                attempt=0,
            )

        self.stdout.write(self.style.SUCCESS(
            f'Expired {count} keys. Webhook tasks dispatched to Celery.
        ))
```

---

### Step 6 — Run the Celery Worker

```
celery -A gamekey_platform worker --loglevel=info
```

In a second terminal, run the command to trigger notifications:

```
python manage.py check_expired_keys
```

Watch the worker terminal — you should see tasks being picked up and executed.

# 🎓 Instructor Teaching Plan

## ⏱ Time Breakdown (90 min)

| Segment | Duration | Activity |
|---|---|---|
| Task queue mental model | 15 min | Diagram: producer (management command) → broker (Redis queue) → worker (Celery process) → result backend (Redis). Compare to a restaurant: the kitchen (producer) puts orders on a ticket rail (broker); the cook (worker) picks them up. The worker runs in a separate process — if the web server goes down, tasks already in the queue are safe. |
| Redis + Celery setup | 15 min | Run Redis via Docker. Write `celery.py`, update `__init__.py`. Add broker/backend URLs to `settings.py`. Verify: `celery -A gamekey_platform inspect ping`. |
| `WebhookDeliveryLog` model | 10 min | Add model, migrate, register in admin. Explain: every delivery attempt (success or failure) is a logged row. This is an audit trail for debugging. |
| Write the Celery task | 25 min | Walk through `@shared_task(bind=True, max_retries=3)`. Explain `bind=True` gives us `self` (the task instance). Show `self.retry(exc=exc, countdown=...)`. Explain exponential backoff: 60s → 120s → 240s. |

| | | |
|---|---|---|
| Update management command | 10 min | Swap synchronous call for `.delay()`. Explain: `.delay()` puts the task in Redis; it returns immediately. The worker picks it up asynchronously. |
| Live demo with worker | 15 min | Two terminals: one runs the worker, one runs `check_expired_keys`. Students watch tasks appear in the worker log and `WebhookDeliveryLog` entries appear in admin. |

## 🌤 Key Concepts to Introduce

- **Task queue** — decouples the producer (who decides what to do) from the worker (who does it). The broker (Redis) is the buffer between them.
- **`@shared_task`** — preferred over `@app.task` because it doesn't require importing the Celery app object. Works with Django's `autodiscover_tasks()`.
- **`bind=True`** — passes the task instance as the first argument (`self`). Required for `self.retry()`.
- **`self.retry(exc, countdown)`** — re-queues the task after `countdown` seconds. Celery tracks the attempt count against `max_retries`.
- **Exponential backoff** — retrying immediately after a failure often fails again (the remote server is still down). Waiting longer each time gives the system time to recover. `60 * (2 ** attempt)` gives 60s, 120s, 240s.
- **`.delay()` vs `.apply_async()`** — `.delay(*args, **kwargs)` is shorthand. `.apply_async(args, kwargs, countdown=30)` gives more control (delays, ETAs, priorities).
- **`CELERY_TASK_ALWAYS_EAGER = True`** — runs tasks synchronously in the same process. Set this in test settings to avoid needing a real worker in tests.

## ⚠ Common Mistakes to Address

- **Not importing `celery_app` in `__init__.py`** — Celery's `autodiscover_tasks` won't run. Tasks exist but are never registered with the worker.
- **Running the management command before starting the worker** — tasks queue up in Redis but nothing processes them. Students see `.delay()` return instantly but nothing happens.
- **Iterating the queryset after `.update()`** — `update()` modifies the DB but not the in-memory queryset objects. Always `list()` the queryset *before* calling `.update()` (already shown in the code — make sure students understand why).
- **`self.retry()` must be `raise self.retry()`** — if you call `self.retry()` without `raise`, the function continues executing after the retry call. Always `raise`.
- **Using `apply()` in production** — `apply()` runs the task synchronously (like `ALWAYS_EAGER`). Only for testing. In production, always use `.delay()` or `.apply_async()`.

## ? Check for Understanding

> "What's in Redis right now, after `.delay()` is called but before the worker picks up the task?"
>
> **Answer:** Redis contains a serialized JSON message (the task payload) representing the Celery task. This message includes the task name, task ID, arguments (like `publisher_id`, `game_key_str`), and other metadata, waiting in a list (acting as a queue).

"What happens if the Celery worker crashes mid-task? Is the task lost?"

**Answer:** With default settings (late acknowledgment disabled), the task is acknowledged when the worker starts, meaning it could be lost if it crashes mid-execution. If late acknowledgment is enabled (`task_acks_late = True`), the task is only acknowledged *after* successful execution; if the worker crashes, the broker (Redis) will re-queue the task for another worker.

"Why do we log *both* successful and failed deliveries to `WebhookDeliveryLog`?"

**Answer:** Logging both creates a complete audit trail. It allows developers to diagnose issues, prove to publishers that webhooks were dispatched, analyze error patterns, and keep track of execution history for debugging delivery problems.

"What does `bind=True` do, and why do we need it for retries?"

**Answer:** `bind=True` binds the task function to the task instance, passing the task instance as the first argument (`self`). We need it because retrying requires calling the `.retry()` method on the task instance itself (`self.retry()`).

"If `max_retries=3` and all retries fail, what happens to the task? How would we know?"

**Answer:** The task will be marked as `FAILURE` by Celery. An exception (`MaxRetriesExceededError`) will be raised and logged. We would know by checking Celery worker logs, monitoring systems, or checking the `WebhookDeliveryLog` in the database where the final log entry will show `success=False` and the final attempt count.

## ✓ Day Checkpoint

Student runs the Celery worker (`celery -A gamekey_platform worker --loglevel=info`) in one terminal and `python manage.py check_expired_keys` in another. The worker terminal shows task receipt and execution. Admin shows new `WebhookDeliveryLog` entries with `success=True` and `response_status=200`.